

tf.Variable Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/Variable

See the [variable guide](<https://tensorflow.org/guide/variable>).

Function Signatures

```
tf.Variable( initial_value=None, trainable=None,
validate_shape=True, caching_device=None, name=None,
variable_def=None, dtype=None, import_scope=None,
constraint=None,
synchronization=tf.VariableSynchronization.AUTO,
aggregation=tf.compat.v1.VariableAggregation.NONE,
shape=None, experimental_enable_variable_lifting=True )
Variable()
```

Code Examples

```
tf.Variable(  
    initial_value=None,  
    trainable=None,  
    validate_shape=True,  
    caching_device=None,  
    name=None,  
    variable_def=None,  
    dtype=None,  
    import_scope=None,  
    constraint=None,  
    synchronization=tf.VariableSynchronization.AUTO,  
    aggregation=tf.compat.v1.VariableAggregation.NONE,  
    shape=None,  
    experimental_enable_variable_lifting=True  
)
```

```
tf.Variable(  
    initial_value=None,  
    trainable=None,  
    validate_shape=True,  
    caching_device=None,  
    name=None,  
    variable_def=None,  
    dtype=None,  
    import_scope=None,  
    constraint=None,  
    synchronization=tf.VariableSynchronization.AUTO,  
    aggregation=tf.compat.v1.VariableAggregation.NONE,  
    shape=None,  
    experimental_enable_variable_lifting=True  
)
```

```
tf.VariableSynchronization.AUTO
```

```
tf.compat.v1.VariableAggregation.NONE
```

```
v = tf.Variable(1.)  
v.assign(2.)  
  
v.assign_add(0.5)
```

```
v = tf.Variable(1.)
```

```
v.assign(2.)
```

Documentation

See the variable guide.

Used in the notebooks

- Migrating model checkpoints
 - Introduction to gradients and automatic differentiation
 - Introduction to Variables
 - Advanced automatic differentiation
 - Better performance with `tf.function`
 - Scalable model compression
 - Custom training loop with Keras and `MultiWorkerMirroredStrategy`
 - Neural style transfer
 - Learned data compression
 - Learnable Distributions Zoo

A variable maintains shared, persistent state manipulated by a program.

The `Variable()` constructor requires an initial value for the variable, which can be a `Tensor` of any type and shape. This initial value defines the type and shape of the variable. After construction, the type and shape of the variable are fixed. The value can be changed using one of the assign methods.

The `shape` argument to `Variable`'s constructor allows you to construct a variable with a less defined shape than its `initial_value`:

Just like any `Tensor`, variables created with `Variable()` can be used as inputs to operations. Additionally, all the operators overloaded for the `Tensor` class are carried over to variables.

When building a machine learning model it is often convenient to distinguish between variables holding trainable model parameters and other variables such as a step variable used to count training steps. To make this easier, the variable constructor supports a `trainable=` parameter. `tf.GradientTape` watches trainable variables by default:

Variables are automatically tracked when assigned to attributes of types inheriting from `tf.Module`.

This tracking then allows saving variable values to training checkpoints, or to `SavedModels` which include serialized `TensorFlow` graphs.

Variables are often captured and manipulated by `tf.functions`. This works the same way the un-decorated function would have:

Variables created inside a `tf.function` must be owned outside the function

and be created only once:

See the `tf.function` documentation for details.

Raises

Attributes

Note that this is different from `initialized_value()` which runs the op that initializes the variable before returning its value. This method returns the tensor that is used by the op that initializes the variable.

Child Classes

class `SaveSliceInfo`

Methods

`assign`

[View source](#)

Assigns a new value to the variable.

This is essentially a shortcut for `assign(self, value)`.

`assign_add`

[View source](#)

Adds a value to this variable.

This is essentially a shortcut for `assign_add(self, delta)`.

assign_sub

[View source](#)

Subtracts a value from this variable.

This is essentially a shortcut for `assign_sub(self, delta)`.

batch_scatter_update

[View source](#)

Assigns `tf.IndexedSlices` to this variable batch-wise.

Analogous to `batch_gather`. This assumes that this variable and the `sparse_delta` `IndexedSlices` have a series of leading dimensions that are the same for all of them, and the updates are performed on the last dimension of indices. In other words, the dimensions should be the following:

```
num_prefix_dims = sparse_delta.indices.ndims - 1
batch_dim = num_prefix_dims + 1
sparse_delta.updates.shape = sparse_delta.indices.shape + var.shape[
    batch_dim:]
```

```
sparse_delta.updates.shape[:num_prefix_dims]
== sparse_delta.indices.shape[:num_prefix_dims]
== var.shape[:num_prefix_dims]
```

And the operation performed can be expressed as:

```
var[i_1, ..., i_n,
```

```
sparse_delta.indices[i_1, ..., i_n, j] = sparse_delta.updates[  
i_1, ..., i_n, j]
```

When `sparse_delta.indices` is a 1D tensor, this operation is equivalent to `scatter_update`.

To avoid this operation one can loop over the first ndims of the variable and using `scatter_update` on the subtensors that result of slicing the first dimension. This is a valid option for `ndims = 1`, but less efficient than this implementation.

count_up_to

[View source](#)

Increments this variable until it reaches limit. (deprecated)

When that Op is run it tries to increment the variable by 1. If incrementing the variable would bring it above limit then the Op raises the exception `OutOfRangeError`.

If no error is raised, the Op outputs the value of the variable before the increment.

This is essentially a shortcut for `count_up_to(self, limit)`.

[View source](#)

In a session, computes and returns the value of this variable.

This is not a graph construction method, it does not add ops to the graph.

This convenience method requires a session where the graph containing this variable has been launched. If no session is passed, the default session is used. See `tf.compat.v1.Session` for more information on launching a graph and on sessions.

experimental_ref

[View source](#)

DEPRECATED FUNCTION

from_proto

[View source](#)

Returns a `Variable` object created from `variable_def`.

gather_nd

[View source](#)

Gather slices from `params` into a `Tensor` with shape specified by `indices`.

See `tf.gather_nd` for details.

get_shape

[View source](#)

Alias of `Variable.shape`.

initialized_value

[View source](#)

Returns the value of the initialized variable. (deprecated)

You should use this instead of the variable itself to initialize another variable with a value that depends on the value of this variable.

[View source](#)

Load new value into this variable. (deprecated)

Writes new value to variable's memory. Doesn't add ops to the graph.

This convenience method requires a session where the graph containing this variable has been launched. If no session is passed, the default session is used. See `tf.compat.v1.Session` for more information on launching a graph and on sessions.

read_value

[View source](#)

Returns the value of this variable, read in the current context.

Can be different from `value()` if it's on another device, with control dependencies, etc.

[View source](#)

Returns a hashable reference object to this Variable.

The primary use case for this API is to put variables in a set/dictionary.

We can't put variables in a set/dictionary as variable.__hash__() is no longer available starting Tensorflow 2.0.

The follo